

СОДЕРЖАНИЕ

Введение.....	7
1 Обзор литературы	8
1.1 Обзор и анализ существующих аналогов	8
1.2 Выбор языка программирования.....	9
1.3 Интегрированная среда разработки	10
1.4 Среда разработки прототипов мобильного приложения	11
1.5 Выбор базы данных.....	11
1.6 Работа с локальными данными через SQLiteOpenHelper	12
1.7 Жизненный цикл Activity	13
1.8 Сетевое взаимодействие с сервером	15
1.9 Нейросетевые алгоритмы рекомендаций	16
1.10 Выводы	17
2 Системное проектирование.....	18
2.1 Архитектура.....	18
2.2 Структурная схема	21
3 Функциональное проектирование	24
3.1 SQLite локальная база данных	24
3.2 Облачная база данных Firebase Firestore	26
3.3 Модуль управления учетной записью.....	29
3.4 Модуль профиля.....	31
3.5 Модуль новостей.....	33
3.6 Модуль фильтрации	34
3.7 Модуль категорий новостей.....	35
3.8 Модуль сохранённых новостей	36
3.9 Модуль нейросетевой рекомендации.....	37
3.10 Модуль рекламы и платной подписки	38
4 Разработка программных модулей	39
5 Программа и методика испытаний.....	40
6 Руководство пользователя.....	41
7 Технико-экономическое обоснование разработки и реализации на рынке мобильного приложения для новостей с поддержкой нейросетевых алгоритмов рекомендаций.....	42
7.1 Характеристика программного средства, разрабатываемого для реализации на рынке.....	42
7.2 Расчёт инвестиций в разработку программного средства	42
7.3 Расчёт экономического эффекта от реализации программного средства на рынке	45
7.4 Расчёт показателей экономической эффективности разработки и реализации программного средства на рынке	47
7.5 Вывод об экономической целесообразности реализации проектного решения	48
Заключение	49
Список использованных источников	50

Приложение А	52
Приложение Б	53
Приложение В.....	54
Приложение Г	55
Приложение Д.....	56
Приложение Е.....	57

ВВЕДЕНИЕ

Потребление новостного контента представляет собой одну из наиболее распространённых форм цифровой активности современного человека. Ежедневно пользователи обращаются к различным источникам информации с целью получения актуальных сведений о событиях в мире, стране и профессиональной сфере. Развитие интернет-технологий, повсеместная доступность мобильных устройств и высокоскоростных сетей передачи данных привели к тому, что процесс получения новостей стал непрерывным и практически мгновенным. В результате объём генерируемого и распространяемого контента постоянно увеличивается, формируя сложную и динамичную информационную среду.

В настоящее время существует множество новостных приложений и платформ, предлагающих различные подходы к агрегации, сортировке и представлению материалов. Тем не менее, стандартные механизмы категоризации и ручной настройки интересов часто оказываются недостаточно гибкими и не учитывают поведенческие особенности конкретного пользователя. В связи с этим возрастает потребность в интеллектуальных системах, способных автоматически анализировать предпочтения, историю просмотров и взаимодействий, а также формировать персонализированную новостную ленту.

В последние годы наблюдается значительное развитие мобильных приложений, и пользователи проводят гораздо больше времени в мобильных интерфейсах, чем на веб-сайтах. Поэтому разработка платформы в формате мобильного приложения является актуальным решением.

Данный дипломный проект посвящен разработке мобильного приложения для новостей с поддержкой нейросетевых алгоритмов рекомендаций.

Целью дипломного проекта является разработка и реализация мобильного приложения, обеспечивающего удобный поиск, агрегацию и интеллектуальную персонализацию новостного контента.

В соответствии с поставленной целью были определены следующие задачи:

- 1 Проведение анализа рынка для изучения конкурентов и определения потребностей целевой аудитории.
- 2 Обоснование стека технологий и выбор программных средств реализации клиентской и серверной частей мобильного приложения.
- 3 Проектирование архитектуры системы и структуры базы данных.
- 4 Разработка и интеграция алгоритмов персонализации и рекомендаций новостного контента.
- 5 Разработка приложения, охватывающая клиентскую и серверную части.
- 6 Проведение тестирования приложения для проверки функциональности и выявления ошибок.

Данный дипломный проект выполнен мной лично, проверен на заимствования, процент оригинальности составляет **XX**% (отчет о проверке на заимствования прилагается).

1 ОБЗОР ЛИТЕРАТУРЫ

1.1 Обзор и анализ существующих аналогов

1.1.1 Google Новости – это агрегатор новостей от корпорации Google, который собирает статьи из различных источников по всему миру. Платформа использует алгоритмы искусственного интеллекта для классификации, группировки и персонализации новостной ленты для каждого пользователя.

Для пользователей приложение предоставляет возможность читать статьи на интересующие темы, настраивать предпочтения и сохранять статьи для последующего прочтения. Ключевой особенностью является мощная система рекомендаций, которая анализирует историю просмотров, сохраненные статьи и явные предпочтения пользователя, чтобы предлагать релевантный контент. Однако алгоритмы работают в основном с актуальной и популярной информацией, в меньшей степени фокусируясь на глубоком анализе личной коллекции сохраненных пользователем материалов [1].

1.1.2 Apple News+ – это платная подписка на новости и журналы от компании Apple. Сервис объединяет сотни изданий и предлагает персонализированные подборки статей. Алгоритм рекомендаций учитывает чтение, лайки и дизлайки, а также подписки пользователя на определенные издания и темы [2].

1.1.3 Flipboard – это социальный агрегатор новостей, который позволяет пользователям создавать персональные «журналы» из статей, видео и подкастов на интересующие их темы. Платформа использует алгоритмы для рекомендации контента на основе выбранных пользователем тем и его социальных подключений [3].

1.1.4 В отличие от Google Новостей и Apple News+, которые фокусируются на актуальной ленте и популярном контенте, и Flipboard, который делает упор на социальное и визуальное взаимодействие, в разрабатываемом приложении «Smart News» планируется сделать акцент на анализе личной коллекции сохраненных пользователем статей.

Главная идея проекта заключается в том, чтобы глубже понять интересы конкретного человека. Специальные нейросетевые алгоритмы изучают тексты из персональной коллекции, находят скрытые связи между темами, повторяющиеся интересы и закономерности поведения. Благодаря этому приложение может не только рекомендовать похожие материалы, но и заранее предугадывать, какая информация окажется полезной в будущем.

Таким образом, «Smart News» переходит от общих рекомендаций к персонализированному подходу, формируя для каждого читателя информационное пространство, основанное на его реальных интересах и привычках чтения. Читатель получает не случайные популярные новости, а контент, который максимально соответствует его индивидуальным интересам.

1.2 Выбор языка программирования

1.2.1 Выбор языка программирования для мобильной разработки. Для мобильной разработки существует несколько вариантов. Языки C/C++ в Android-разработке применяются в случаях, когда критически важны высокая производительность, низкоуровневый доступ к аппаратным ресурсам и полный контроль над управлением памятью – например, при создании игровых движков или вычислительно сложных системных приложений. Однако для данного проекта подобные требования не актуальны, поэтому выбор будет сделан в пользу более современных и лаконичных языков, которые ускоряют разработку и упрощают создание пользовательского интерфейса.

Kotlin – это кроссплатформенный, статически типизированный язык программирования, работающий на платформе JVM и полностью совместимый с Java. Он сочетает лучшие черты объектно-ориентированного и функционального программирования, предлагая выразительный и лаконичный синтаксис. По сравнению с Java, Kotlin позволяет решать те же задачи с меньшим объёмом кода, что повышает читаемость и поддерживаемость проекта. Кроме того, язык предоставляет современные средства разработки, такие как корутины для асинхронных операций, расширения функций и nullable-безопасность на уровне системы типов [4].

Хотя Java исторически является одним из основных языков Android-разработки и обладает развитой экосистемой, для дипломного проекта будет выбран Kotlin. Это решение обусловлено его современностью, лаконичностью и ориентацией на безопасность кода. Kotlin полностью совместим с существующими Java-библиотеками и инструментами, что позволяет использовать все возможности экосистемы Android. Кроме того, официальная поддержка Google и активное сообщество разработчиков делают Kotlin надёжным и перспективным выбором для создания качественных приложений [5]. Наличие опыта работы с этим языком также стало важным фактором, который позволит уверенно реализовать все требуемые функции проекта.

1.2.2 Выбор языка программирования для серверной разработки будет сделан в пользу Python и современного веб-фреймворка FastAPI. Данное решение является наиболее сбалансированным по сравнению с классическими серверными решениями на базе Java (Spring) или PHP (Laravel), так как указанная связка позволяет достичь высокой производительности без избыточного усложнения кодовой базы. Python и FastAPI обладают рядом существенных преимуществ [6]:

- 1 Высокая скорость разработки благодаря простому и читаемому синтаксису Python.

- 2 Встроенная поддержка асинхронности, обеспечивающая высокую производительность при обработке множества запросов.

- 3 Автоматическая валидация данных и удобная работа с моделями через Pydantic.

4 Автоматическая генерация документации API (Swagger/OpenAPI).

5 Большое количество библиотек для машинного обучения и анализа данных (NumPy, PyTorch, scikit-learn), что особенно важно для реализации рекомендательной системы.

6 Легкость интеграции с внешними сервисами (например, NewsAPI [7]) и другими технологиями.

7 Активное сообщество и постоянная поддержка.

В совокупности эти факторы делают Python и FastAPI оптимальным выбором для разработки серверной части рекомендательного сервиса, обеспечивая высокую производительность, масштабируемость, простоту сопровождения и удобство дальнейшего расширения функциональности приложения.

1.3 Интегрированная среда разработки

Android Studio является официальной интегрированной средой разработки для платформы Android, разработанной на основе IntelliJ IDEA. В отличие от универсальных редакторов кода, таких как Visual Studio Code, Android Studio предлагает глубокую интеграцию с Android SDK. В то время как VS Code требует установки множества сторонних плагинов для отладки и сборки, Android Studio предоставляет готовую экосистему, включая продвинутые профилировщики памяти и инструменты анализа APK-файлов [8]. Использование данной среды позволит существенно повысить продуктивность разработки за счет интеллектуального автозавершения кода, инструментов рефакторинга и встроенной системы сборки Gradle. Это делает Android Studio более предпочтительным выбором по сравнению с устаревшими решениями (например, Eclipse), поддержка которых давно прекращена.

1.3.1 Встроенный эмулятор в Android Studio позволяет тестировать приложения на различных версиях Android и моделях устройств, не имея физического телефона. Это особенно полезно для проверки совместимости и поведения приложения на разных экранах и конфигурациях.

Чтобы настроить эмулятор надо открыть AVD Manager, перейти в меню Tools, выбрать AVD Manager (Android Virtual Device Manager) и нажать на кнопку Create Virtual Device. В настройках можно указать различные параметры устройства и проверить работоспособность приложения для разных моделей.

1.3.2 UI-дизайнер в Android Studio – это визуальный редактор, который позволяет создавать пользовательские интерфейсы для приложений.

При разработке xml-файла макета можно выбрать разные режимы Design, Code и Split. Например, в Design режиме элементы можно создавать в визуальном редакторе. В левой части окна находится панель инструментов с элементами интерфейса (кнопки, текстовые поля, изображения и т.д.). Для

добавленного элемента отображаются все возможные его настройки и свойства. В режиме Design сразу видно, как будет выглядеть интерфейс, что значительно упрощает процесс разработки.

1.4 Среда разработки прототипов мобильного приложения

Разработка мобильного приложения как продукта обычно состоит из следующих этапов:

- 1 Анализ рынка и продукта.
- 2 Составление плана разработки мобильного приложения.
- 3 Прототипирование.
- 4 Разработка.
- 5 Тестирование.
- 6 Запуск и публикация мобильного приложения.
- 7 Поддержка продукта.

В данном проекте разработка дизайна и прототипов интерфейса будет осуществляться непосредственно в среде Android Studio. Ключевым преимуществом данного подхода станет использование визуального редактора Layout Editor, который предоставит палитру компонентов с готовыми элементами Material Design, а также возможность предпросмотра макетов на различных устройствах и при разных ориентациях экранов. Для описания интерфейсов будет применяться язык разметки XML, что обеспечит точное соответствие между дизайном и его программной реализацией [9].

Выбор данного инструментария обусловлен возможностью интеграции процесса проектирования интерфейса с этапом разработки, что ускоряет процесс реализации и обеспечивает согласованность дизайна. Кроме того, работа в среде Android Studio исключает необходимость использования дополнительных инструментов и синхронизации между ними, так как весь процесс от проектирования до реализации происходит в едином рабочем пространстве.

1.5 Выбор базы данных

1.5.1 Категории данных в приложении. В мобильном приложении будут использоваться следующие категории данных:

- 1 Локальные данные пользователя.
- 2 Облачные данные для синхронизации между устройствами.

Локальные данные пользователя – это данные, которые должны быть всегда доступны, в том числе при отсутствии интернет-соединения. К ним относятся профиль пользователя, сохранённые новости, пользовательские предпочтения. Эти данные должны обрабатываться быстро, храниться непосредственно на устройстве и обеспечивать возможность поиска и фильтрации. В связи с этим для их хранения используется локальная реляционная база данных.

Облачные данные предназначены для синхронизации пользовательских данных между разными устройствами одного пользователя. Они обеспечивают сохранность данных при смене устройства, повторной установке приложения и работе с несколькими устройствами одновременно. Данные хранятся на удалённом сервере и синхронизируются с локальным хранилищем при наличии сетевого подключения.

Такое разделение позволяет обеспечить автономную работу приложения, высокую производительность и одновременно поддержку облачной синхронизации.

1.5.2 В соответствии с архитектурой приложения и обрабатываемой информации, в проекте будут использоваться две дополняющие друг друга базы данных.

Для хранения данных непосредственно на устройстве пользователя будет применяться SQLite. В отличие от серверных СУБД, таких как PostgreSQL или MySQL, SQLite не требует отдельного процесса сервера, так как работает напрямую с файлом внутри приложения. Это делает её стандартом для мобильной разработки. Использование SQLite обеспечит высокую скорость доступа к данным и поддержку сложных SQL-запросов для фильтрации информации. Это позволит приложению сохранять полную функциональность при отсутствии интернет-соединения.

Для централизованного хранения и репликации данных между различными устройствами одного пользователя будет задействована облачная платформа Firebase [10]. По сравнению с PostgreSQL на выделенном сервере использование Firebase значительно сократит время разработки, так как платформа предоставляет готовые SDK для автоматической синхронизации данных. Настройка собственного API для связи с PostgreSQL потребовала бы значительно больших трудозатрат на реализацию логики обработки конфликтов при одновременном изменении данных.

1.6 Работа с локальными данными через SQLiteOpenHelper

Для хранения локальных данных пользователя, таких как профиль, сохранённые новости и предпочтения, используется SQLite в сочетании с классом SQLiteOpenHelper – стандартным решением Android для работы с реляционной базой данных.

SQLiteOpenHelper предоставляет абстракцию над SQLite и упрощает работу с базой данных в Android-приложениях, управляя созданием и обновлением схемы базы данных.

1.6.1 Entity представляют собой классы данных, которые отображаются на таблицы в реляционной базе данных. Каждый объект entity соответствует строке в таблице, а его поля – столбцам. Одна из реализаций – это data-классы, которые содержат всю необходимую информацию о структуре данных, включая первичные ключи, внешние ключи и отношения между таблицами.

Для каждой сущности определяются соответствующие поля данных, которые включают уникальные идентификаторы, текстовые поля для хранения информации, числовые значения для хранения состояний и флагов, а также поля для хранения связей между различными таблицами. Структура сущностей обеспечивает целостность данных и позволяет эффективно организовывать информацию в реляционной модели.

1.6.2 Database Helper является центральным компонентом для управления базой данных, который наследуется от `SQLiteOpenHelper`. Этот класс отвечает за создание, обновление и управление структурой базы данных, а также предоставляет API для выполнения операций с данными.

В `Database Helper` инкапсулируется вся логика работы с базой данных, включая:

- 1 Создание таблиц при инициализации базы данных.
- 2 Обработку миграций данных при обновлении версии базы.
- 3 Выполнение операций CRUD (Create, Read, Update, Delete).
- 4 Управление транзакциями для обеспечения целостности данных.
- 5 Обработку ошибок и ведение логов операций.

Класс предоставляет методы для работы с пользователями, их предпочтениями и сохраненными данными, обеспечивая удобный интерфейс для взаимодействия с persistence-слоем приложения.

1.6.3 Для обеспечения синхронизации данных между локальным хранилищем и облачной базой данных будет реализован механизм двусторонней репликации. Этот механизм обеспечивает согласованность различных решений для хранения данных и позволяет пользователям работать с приложением как онлайн, так и офлайн.

Синхронизация осуществляется через специализированный облачный сервис, который предоставляет API для обмена данными. Локальная база данных сохраняет копию соответствующих данных и автоматически синхронизирует изменения при наличии сетевого подключения. Реализация включает:

- 1 Отслеживание изменений в локальной базе данных.
- 2 Пакетную отправку изменений в облачное хранилище.
- 3 Разрешение конфликтов при расхождении данных.
- 4 Фоновую синхронизацию без прерывания работы пользователя.
- 5 Механизм повторных попыток при сетевых сбоях.

1.7 Жизненный цикл Activity

Понимание жизненного цикла `Activity` является фундаментальным для разработки эффективных и надежных Android-приложений. `Activity` отвечает за отображение контента и взаимодействие с пользователем. Неправильное управление жизненным циклом `Activity` может привести к потере данных, утечкам памяти, сбоям в работе приложения.

Жизненный цикл Activity представляет собой набор состояний, через которые проходит Activity в течение своего существования. Каждое состояние соответствует определенному этапу работы Activity, и в каждом состоянии Activity может выполнять определенные действия. Переходы между состояниями жизненного цикла происходят в ответ на различные события, такие как запуск Activity, переключение между приложениями, поворот экрана, получение входящего звонка и т.д.

Понимание жизненного цикла Activity позволяет разработчикам:

1 Правильно управлять ресурсами. Освобождать ресурсы, когда Activity неактивна, и выделять их, когда Activity активна, чтобы избежать утечек памяти и повысить производительность приложения.

2 Сохранять и восстанавливать данные. Сохранять состояние Activity при переключении между приложениями или повороте экрана, чтобы пользователь не терял данные и не начинал работу с приложением заново.

3 Обращивать прерывания. Правильно реагировать на прерывания, такие как входящие звонки или SMS-сообщения, чтобы приложение продолжало работать корректно после прерывания.

4 Оптимизировать производительность. Выполнять ресурсоемкие операции только тогда, когда Activity активна, чтобы не замедлять работу приложения в фоновом режиме.

Состояний жизненного цикла Activity и переходов между ними представлено на рисунке 1.1.

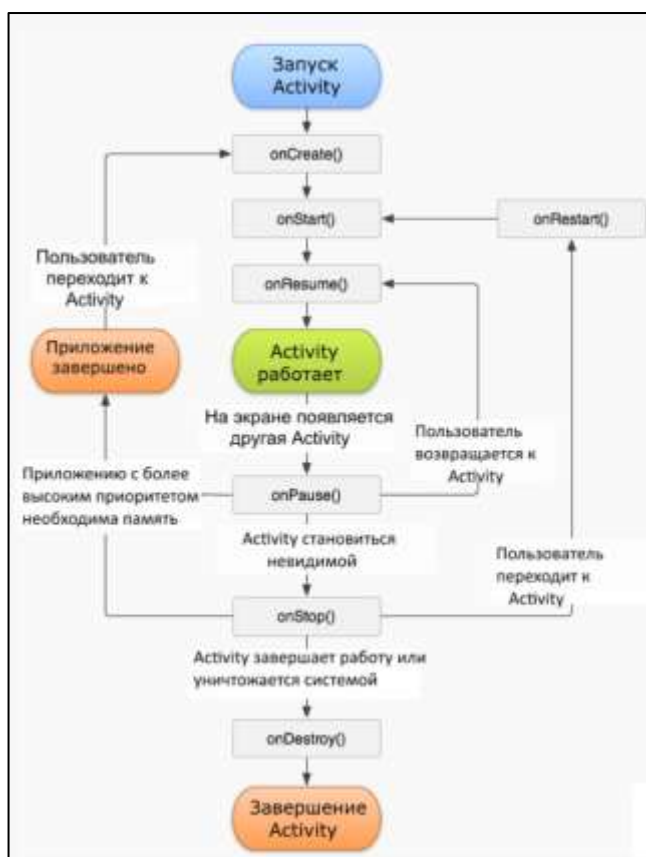


Рисунок 1.1 – Жизненный цикл Activity [11]

1.8 Сетевое взаимодействие с сервером

В современной разработке Android-приложений, взаимодействие с серверами является критически важным для обеспечения динамичного контента, пользовательской функциональности и обмена данными. Для реализации взаимодействия мобильного приложения с сервером будет использоваться комбинация технологий: REST API, Retrofit и OkHttp. Выбор данного стека обусловлен его высокой производительностью, удобством разработки и широкой распространённостью в экосистеме Android. В отличие от низкоуровневых средств работы с сетью, требующих значительного объёма ручной реализации, а также более простых библиотек, ориентированных преимущественно на небольшие запросы, выбранное решение позволит обеспечить лучшую масштабируемость, читаемость кода и простоту сопровождения проекта при увеличении количества серверных операций.

1.8.1 REST API – это архитектурный стиль, который определяет правила обмена данными между клиентом и сервером. Эти правила предоставляет сервер, а мобильное приложение (клиент) использует этот интерфейс для взаимодействия с ним. Приложение отправляет HTTP-запросы на определенные конечные точки REST API, чтобы получить данные или выполнить какие-либо действия на сервере. Сервер принимает HTTP-запросы от клиентов, обрабатывает их и возвращает ответы в определенном формате [12]. На практике данный стиль определяется конечные точки (эндпоинты), то есть определяет набор URL-адресов, которые представляют ресурсы, доступные на сервере и использует стандартные HTTP-методы для выполнения операций над ресурсами, а именно:

1 OkHttp.

2 Retrofit.

Использование REST-подхода по сравнению с альтернативами, такими как SOAP или собственные протоколы обмена, позволит упростить архитектуру системы и снизить сетевые издержки за счёт передачи данных в лёгком формате JSON. Это обеспечит более высокую скорость взаимодействия и упростит интеграцию мобильного клиента с серверной частью, а также облегчит последующее расширение функциональности приложения.

1.8.2 OkHttp – это эффективный HTTP-клиент для Java и Android. OkHttp используется в Android-приложении для отправки HTTP-запросов на сервер и получения ответов. OkHttp автоматически управляет соединениями, повторно используя их для повышения производительности и снижения задержек. По сравнению с использованием стандартного класса `URLConnection`, который требует ручной настройки соединений, потоков и обработки исключений, OkHttp значительно упрощает реализацию сетевого слоя и уменьшает вероятность ошибок, связанных с некорректным управлением ресурсами.

Retrofit – это популярная библиотека для работы с сетевыми запросами в Android-приложениях, разработанная компанией Square. Она предназначена для упрощения взаимодействия с RESTful API, предоставляя удобный способ отправки HTTP-запросов и обработки ответов. По сравнению с библиотекой Volley, которая лучше подходит для простых и кратковременных запросов, Retrofit обеспечивает более гибкую настройку, удобную работу с моделями данных и лучшую интеграцию со сложными REST-сервисами. Это делает его более подходящим выбором для приложения с большим количеством API-запросов и развитой серверной логикой. Схема работы библиотеки Retrofit в Android представлена на рисунке 1.2.

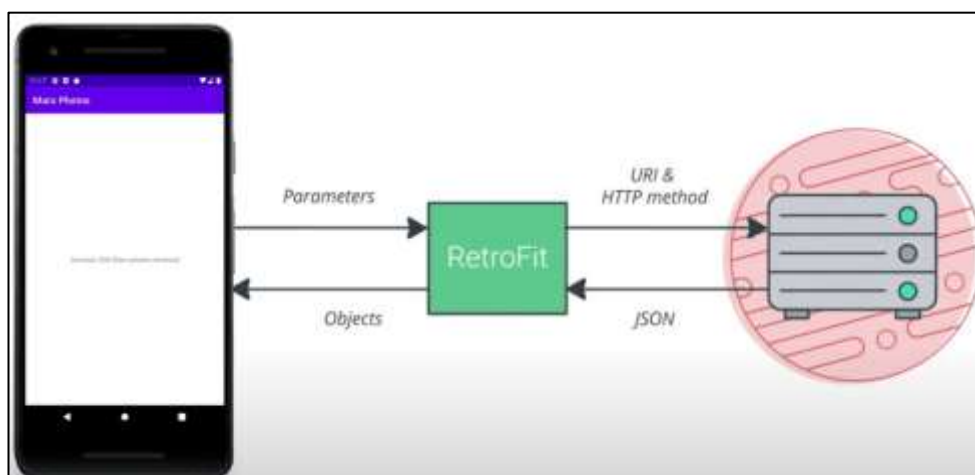


Рисунок 1.2 – Схема работы библиотеки Retrofit в Android [13]

1.9 Нейросетевые алгоритмы рекомендаций

Для реализации ключевой функции приложения – интеллектуального анализа персональной коллекции статей будут использованы современные алгоритмы обработки естественного языка (NLP). Выбор конкретных архитектур нейросетей обусловлен необходимостью находить семантические связи между текстами, а не просто искать совпадения по ключевым словам.

В процессе разработки будет сделан выбор в пользу контентной фильтрации, которая станет основным методом построения рекомендаций. В отличие от популярной коллаборативной фильтрации, требующей огромного массива данных о действиях тысяч других пользователей, контентно-ориентированный подход позволит системе эффективно работать с личной коллекцией конкретного человека. Это решит проблему «холодного старта» и обеспечит высокую точность рекомендаций даже в тех случаях, когда база пользователей приложения только начинает формироваться.

Технологической основой для извлечения смысловых признаков из статей послужит метод векторных представлений слов. По сравнению с классическими статистическими моделями, такими как TF-IDF или «мешок слов», которые учитывают лишь частоту появления конкретных терминов, нейросетевые модели типа Word2Vec или FastText позволяют системе

оперировать значениями и контекстом. Это обеспечит понимание сходства между материалами, написанными разными словами, но на одну и ту же тему. В качестве наиболее перспективного решения будет рассматриваться применение архитектуры Transformer, в частности моделей семейства BERT. Использование трансформеров станет более эффективным решением по сравнению с рекуррентными нейронными сетями, так как они способны одновременно анализировать весь текст статьи, учитывая сложные логические связи между предложениями.

1.10 Выводы

В результате проведённого анализа для разработки мобильного приложения «Smart News» будет выбран современный и сбалансированный стек технологий, который обеспечит высокую производительность, удобство разработки и возможность дальнейшего масштабирования системы. В качестве основного инструмента создания Android-клиента будет использоваться язык программирования Kotlin и среда разработки Android Studio, что позволит упростить реализацию пользовательского интерфейса, повысить читаемость кода и сократить время разработки.

Серверная часть будет реализована на языке Python с применением фреймворка FastAPI, которые обеспечат высокую скорость создания API, поддержку асинхронной обработки запросов и удобную интеграцию с библиотеками анализа данных и машинного обучения. Для хранения данных на устройстве пользователя будет применяться SQLite, что обеспечит автономную работу приложения без подключения к сети, а для синхронизации и централизованного хранения информации будет использоваться облачная платформа Firebase.

Взаимодействие мобильного клиента с сервером будет осуществляться с помощью библиотек Retrofit и OkHttp, которые позволят эффективно выполнять сетевые запросы и обеспечат стабильность передачи данных. Для реализации интеллектуальных рекомендаций будут применяться методы обработки естественного языка и нейросетевые модели, включая архитектуру BERT, что позволит анализировать тексты статей на семантическом уровне и формировать персонализированные подборки материалов.

Таким образом, выбранные технологии будут способствовать созданию надёжного, масштабируемого и интеллектуального сервиса, который будет предоставлять пользователю максимально релевантный и индивидуально подобранный новостной контент.

2 СИСТЕМНОЕ ПРОЕКТИРОВАНИЕ

В этом разделе на основе системного подхода определяется обобщенная структура программы, описывается назначение выделенных блоков, а также связи между ними. Для начала рассмотрим архитектуру приложения.

2.1 Архитектура

Архитектура, представленная на рисунке 2.1, определяет, как компоненты приложения взаимодействуют между собой и с внешними системами.

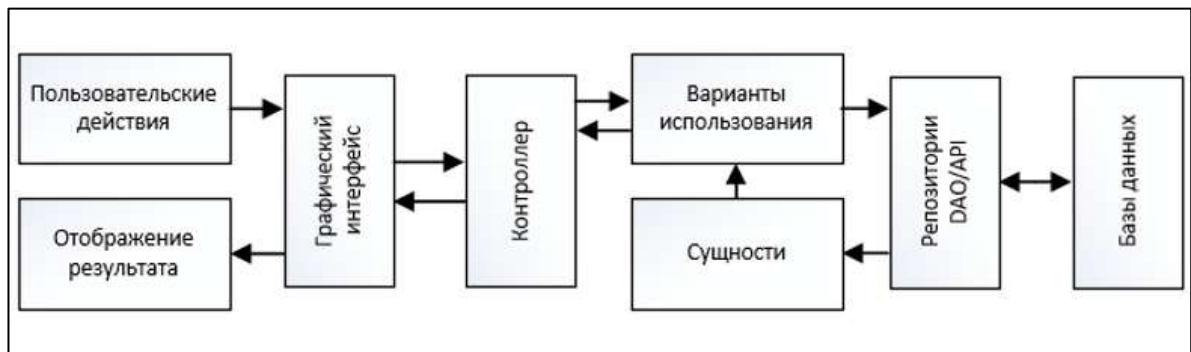


Рисунок 2.1 – Архитектура приложения

Графический интерфейс, контроллер, варианты использования и сущности вместе реализуют MVC шаблон.

MVC (Model-View-Controller) – это архитектурный шаблон проектирования, который разделяет приложение на три основных компонента:

- 1 Model: данные и бизнес-логика.
- 2 View: отображение данных и взаимодействие с пользователем.
- 3 Controller: посредник между Model и View.

Взаимодействие пользователя с приложением происходит по четко определенному циклу. Когда пользователь выполняет какое-либо действие, например, нажимает на кнопку или вводит текст, это действие сначала попадает в графический интерфейс.

Графический интерфейс, выступая в роли View, не обрабатывает это действие самостоятельно, а передает его контроллеру для дальнейшей обработки. Это обеспечивает соблюдение принципа единственной ответственности, согласно которому каждый компонент системы должен решать только одну задачу.

После того как контроллер получил информацию о действии пользователя, он взаимодействует с моделью, которая содержит всю бизнес-логику приложения.

Модель обрабатывает полученный запрос, выполняет необходимые вычисления или операции с данными, и возвращает результат обратно контроллеру. Контроллер, в свою очередь, передает эти данные в графический

интерфейс, который обновляет отображение, показывая пользователю результат выполнения его действия.

Такой подход обеспечивает четкое разделение ответственностей между компонентами системы. Графический интерфейс отвечает только за отображение данных и сбор пользовательского ввода, модель содержит бизнес-логику и работу с данными, а контроллер управляет потоком информации между ними. Это делает систему более гибкой, масштабируемой и простой в поддержке.

Принцип MVC-шаблона изображен на рисунке 2.2.

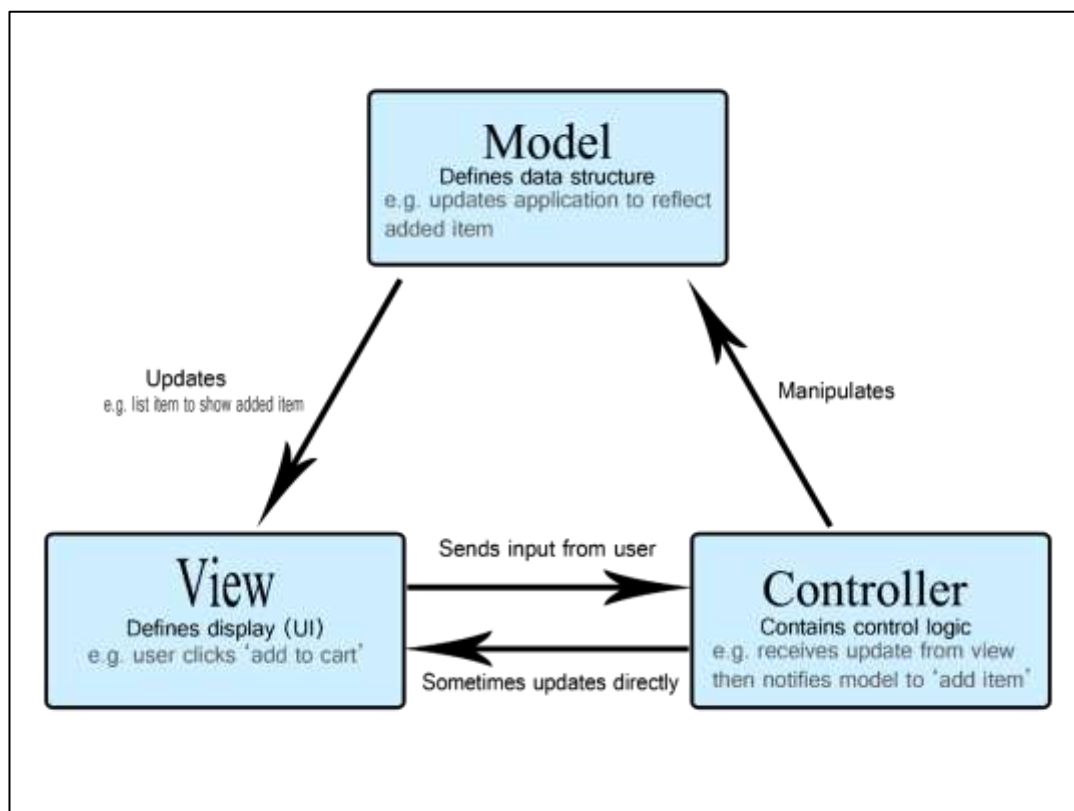


Рисунок 2.2 – Принцип MVC-шаблона [14]

2.1.1 Блоки пользовательских действий и отображения результата. Данные два блока обеспечивают взаимодействие пользователя с приложением. Они включают в себя ввод данных пользователем (пользовательские действия) и вывод результатов обратно пользователю (отображение результата).

2.1.2 Графический интерфейс представляет собой View из шаблона MVC. Блок графического интерфейса визуализирует данные, преобразуя их в удобный для пользователя формат и отображает на экране в виде текста, изображения, кнопок и других элементов. А при изменении в ходе работы приложения обновляет интерфейс по указанию контроллера. Также этот блок принимает действия от пользователя (например, нажатия кнопок, ввод текста) и передает его контроллеру для дальнейшей обработки. Блок графического

интерфейса не содержит никакой логики, его основная задача – собирать пользовательские действия и отображать данные.

Графический интерфейс описывается в XML-файлах, которые определяют структуру, внешний вид пользовательского интерфейса. Если требуется в файле указываются обработчики нажатий на элементы и другие дополнительные настройки.

2.1.3 Контроллер в архитектуре MVC играет ключевую роль посредника между графическим интерфейсом и моделью данных. В приложении функции контроллера выполняют классы Activity. Эти классы получают пользовательские события от графического интерфейса, принимают решения о том, как обработать эти события, и взаимодействуют с моделью для выполнения необходимых операций.

Когда пользователь совершает какое-либо действие, например, нажимает на кнопку «Войти», соответствующее событие сначала попадает в контроллер. Контроллер анализирует это событие и определяет, какие действия необходимо выполнить. В случае с кнопкой «Войти» контроллер может проверить введенные данные, затем обратиться к модели для аутентификации пользователя, и наконец, в зависимости от результата, обновить графический интерфейс.

Важной особенностью контроллера является то, что он не содержит бизнес-логики приложения. Вся логика обработки данных сосредоточена в модели. Контроллер лишь управляет потоком информации между графическим интерфейсом и моделью, принимая решения о том, какие методы модели вызвать в ответ на те или иные пользовательские действия.

Кроме обработки пользовательских событий, контроллер также отвечает за управление жизненным циклом экранов приложения, обработку системных событий (например, изменение ориентации устройства), и навигацию между различными экранами. Все это делает контроллер центральным узлом в архитектуре приложения, который координирует работу всех остальных компонентов.

2.1.4 Модель в архитектуре MVC представляет собой ядро приложения, содержащее всю бизнес-логику и данные. В данном проекте модель разделена на две основные составляющие: сущности (entities) и варианты использования (use cases).

Сущности – это объекты данных, которые представляют ключевые концепции предметной области приложения. Каждая сущность содержит данные (поля класса) и минимальную логику их обработки (методы класса). Сущности не зависят от графического интерфейса или контроллера, что позволяет использовать их в разных частях приложения. Варианты использования описывают конкретные сценарии взаимодействия пользователя с системой. Варианты использования инкапсулируют сложную бизнес-логику, скрывая детали реализации от контроллера. Это позволяет изменять логику работы приложения, не затрагивая другие компоненты.

Модель также включает в себя механизмы для хранения и извлечения данных. В данном проекте используются два типа хранилищ: локальная база данных SQLite для хранения данных на устройстве, и удаленная база данных Firebase для хранения данных на сервере. Доступ к этим хранилищам осуществляется через специальные классы-репозитории, которые предоставляют унифицированный интерфейс для работы с данными, независимо от их физического расположения.

2.1.5 Репозиторий – это абстракция, которая обеспечивает удобный доступ и управление данными, предоставляя интерфейс для взаимодействия с различными источниками, такими как базы данных и внешние API.

В проекте используются два типа баз данных: локальная база данных SQLite и облачная база данных Firebase.

Локальная база данных SQLite хранит информацию о пользователях и сохраненных новостях, поддерживая основные операции: создание, чтение, обновление и удаление данных. Это обеспечивает быстрый доступ к данным даже без подключения к интернету.

Облачная база данных Firebase Firestore используется для синхронизации данных пользователей и сохраненных новостей между устройствами. Данные надежно сохраняются в облаке, что гарантирует их доступность и целостность.

Для получения новостей из внешнего источника используется API, реализованное с помощью библиотеки Retrofit. Это позволяет запрашивать актуальные данные, такие как новости по категориям, преобразовывая сетевые запросы в удобные методы.

Такой подход к работе с данными обеспечивает гибкость, масштабируемость и надежность приложения, позволяя эффективно управлять локальными и удаленными данными.

2.2 Структурная схема

Приложение имеет модульную систему, где модуль является изолированной частью приложения, которая имеет свои собственные интерфейсы и логику. Так как проект отображает как клиентскую, так и серверную части, то модуль является более широкой концепцией, обозначающая независимую часть системы, которая может содержать одну или несколько моделей, а также другие компоненты, такие как контроллеры и представления.

Основная задача модуля – группировка связанных компонентов для обеспечения четкой структуры, поэтому деление на модули основывается на назначении выделенных блоков.

Система состоит из следующих модулей:

- 1 Модуль управления учетной записью.
- 2 Модуль рекламы и платной подписки.
- 3 Модуль профиля.

- 4 модуль новостей.
- 5 Модуль фильтрации.
- 6 Модуль категорий новостей.
- 7 Модуль сохраненных новостей.
- 8 Модуль нейросетевой рекомендации.

Структурная схема, иллюстрирующая перечисленные блоки и связи между ними приведена на чертеже ГУИР.400201.124 С1.

2.2.1 Модуль управления учетной записью обеспечивает процессы регистрации, аутентификации и авторизации пользователей в системе. Пользователь может создать учетную запись, указав логин, адрес электронной почты и пароль, а также выполнить вход в приложение с использованием сохранённых данных.

Модуль поддерживает следующие функции: регистрация нового пользователя, вход в систему, выход из аккаунта, восстановление и изменение пароля, хранение и обновление персональных данных пользователя. Данные учетных записей синхронизируются с облачной базой данных Firebase, что обеспечивает их сохранность и доступность на разных устройствах.

2.2.2 Модуль рекламы и платной подписки отвечает за монетизацию приложения. Он обеспечивает отображение рекламных материалов для пользователей бесплатной версии и управление платной подпиской, позволяющей отключить рекламу и получить дополнительные функции.

Модуль реализует следующие возможности: показ рекламных объявлений, проверка статуса подписки, оформление и продление подписки, хранение информации о платежах и ограничениях доступа. При активной подписке реклама отключается, а пользователь получает расширенный функционал приложения.

2.2.3 Модуль профиля предназначен для хранения и управления персональной информацией пользователя. Он содержит данные пользователя, такие как имя, электронная почта, настройки приложения.

Пользователь может выполнять следующие действия: просмотр и редактирование личных данных, изменение настроек. Модуль взаимодействует с локальным и облачным хранилищами для обеспечения актуальности данных.

2.2.4 Модуль новостей является основной функциональной частью приложения и отвечает за получение, хранение и отображение новостного контента. Новости загружаются из внешнего API, обрабатываются и отображаются пользователю в удобном виде. Модуль реализует следующие функции: получение списка новостей, обновление контента, отображение полной информации о статье, загрузка изображений, кэширование данных для офлайн-доступа. Для выполнения сетевых запросов используется библиотека

Retrofit, обеспечивающая преобразование HTTP-запросов в методы приложения.

2.2.5 Модуль фильтрации обеспечивает сортировку и отбор новостей по заданным критериям. Он позволяет пользователю настраивать отображаемый контент в соответствии со своими интересами.

Поддерживаются следующие параметры фильтрации: категории, дата публикации, индикатор настраивание статей. Применение фильтров позволяет формировать персонализированную ленту новостей и упрощает поиск необходимой информации.

2.2.6 Модуль категорий новостей отвечает за структурирование контента по тематическим разделам. Каждая новость относится к определённой категории (например: политика, спорт, технологии, культура).

Модуль предоставляет пользователю возможность просматривать новости по выбранной категории, переключаться между разделами и получать тематические подборки. Это упрощает навигацию по приложению и повышает удобство использования.

2.2.7 Модуль сохранённых новостей предназначен для хранения статей, выбранных пользователем для последующего просмотра. Пользователь может добавлять новости в список сохранённых, удалять их и просматривать в любое время.

Данные сохраняются локально в базе данных SQLite, что обеспечивает быстрый доступ и возможность работы без подключения к интернету. При наличии сети информация может синхронизироваться с облачным хранилищем.

2.2.8 Модуль нейросетевой рекомендации предназначен для персонализации новостной ленты на основе интересов пользователя. Он анализирует сохранённые статьи.

На основе полученных данных формируются рекомендации, представляющие собой список новостей, наиболее соответствующих предпочтения пользователя.

3 ФУНКЦИОНАЛЬНОЕ ПРОЕКТИРОВАНИЕ

Основываясь на структурной схеме и архитектуре приложения, рассмотрим детально функционирование отдельных модулей и приложения в целом. Каждый функциональный модуль имеет схожую внутреннюю структуру, включающую следующие ключевые компоненты:

1 Activity – компонент, реализующий роль контроллера в рамках паттерна MVC. Activity отвечает за обработку пользовательских событий, управление жизненным циклом экрана, навигацию между экранами и взаимодействие с моделью (репозиторием и сущностями). Через Activity осуществляется передача данных между пользовательским интерфейсом и бизнес-логикой.

2 Adapter – класс, обеспечивающий связь между набором данных и элементами пользовательского интерфейса, такими как RecyclerView или ListView. Адаптер создаёт и настраивает представления для каждого элемента данных, обновляет интерфейс при изменении данных, а также обрабатывает пользовательские взаимодействия (например, клики) с элементами списка.

3 Сущности (Entities) – data-классы, представляющие объекты предметной области (пользователь, сохранённая новость и т.д.). Они содержат поля, соответствующие атрибутам таблиц базы данных, и используются для передачи данных между слоями приложения.

4 Репозиторий (DatabaseHelper) – слой доступа к данным, построенный на основе SQLiteOpenHelper. Репозиторий инкапсулирует логику выполнения операций с локальной базой данных (SQLite), а также обеспечивает синхронизацию с облачным хранилищем Firebase. Через репозиторий осуществляются все CRUD-операции с пользователями и сохранёнными новостями.

Данная унифицированная структура обеспечивает ясность и согласованность архитектуры приложения, построенной по паттерну MVC, и облегчает его поддержку и дальнейшее развитие. Дополнительно следует отметить, что взаимодействие между модулями приложения осуществляется через четко определённые интерфейсы и методы, что обеспечивает слабую связанность компонентов системы и повышает её модульность.

Для описания функционального проектирования целесообразно структурировать материал в соответствии с модулями, выделенными на этапе системного проектирования.

Диаграмма классов, иллюстрирующая состав и взаимосвязи классов каждого модуля, представлена на чертеже ГУИР.400201.124 РР.1. Ниже приводится описание классов по каждому модулю.

3.1 SQLite локальная база данных

Так как приложение имеет модульную структуру, база данных является общим ресурсом, доступным для всех модулей. Как было указано ранее, в проекте используются две системы хранения данных: локальная реляционная

база данных SQLite и облачное Firebase Firestore. SQLite представляет собой встроенную базу данных, которая размещается непосредственно на устройстве пользователя и обеспечивает автономную работу приложения, быстрый доступ к данным и возможность выполнения сложных запросов.

В приложении SQLite реализована с использованием стандартного для Android класса `SQLiteOpenHelper`, который управляет созданием, обновлением и версионированием базы данных, а также предоставляет удобные методы для выполнения операций с таблицами. Локальная база данных хранит две основные категории информации: данные пользователей (учётные записи, настройки, список предпочтительных категорий) и сохранённые пользователем новости (избранное). Каждая таблица представлена в виде соответствующей сущности (Entity) – data-класса, поля которого соответствуют столбцам таблицы. Работа с данными осуществляется через класс-репозиторий `DatabaseHelper`, который инкапсулирует все SQL-запросы и обеспечивает взаимодействие с остальными модулями приложения.

Ниже представлено детальное описание таблиц локальной базы данных, их атрибутов, типов данных и связей между ними.

3.1.1 Таблица 3.1 предназначена для хранения информации о зарегистрированных пользователях приложения. Каждая запись соответствует одному пользователю. Поле `id` является первичным ключом и автоматически генерируется при вставке новой записи. Поле `email` имеет ограничение уникальности, так как адрес электронной почты используется для идентификации пользователя при аутентификации и синхронизации с облаком.

Таблица 3.1 – Таблица `Users`

Атрибут	Тип	Описание
<code>id</code>	INTEGER	Уникальный идентификатор пользователя (первичный ключ, автоинкремент)
<code>name</code>	TEXT	Имя пользователя
<code>email</code>	TEXT	Адрес электронной почты
<code>password</code>	TEXT	Хеш пароля
<code>news_categories</code>	TEXT	Список предпочитаемых категорий новостей, разделённых запятыми
<code>is_vip</code>	INTEGER	Флаг VIP-статуса: 1 – подписка активна, 0 – бесплатный пользователь

3.1.2 Таблица 3.2 хранит новости, добавленные пользователем в избранное для последующего прочтения в офлайн-режиме. Каждая запись представляет одну сохранённую статью. Поле `id` является первичным ключом. Поле `email` выступает внешним ключом, ссылающимся на поле `email` таблицы `Users`, и определяет принадлежность сохранённой новости конкретному пользователю.

Таблица 3.2 – Таблица SavedNews

Атрибут	Тип	Описание
id	INTEGER	Уникальный идентификатор сохранённой новости (первичный ключ, автоинкремент)
email	TEXT	Email пользователя, которому принадлежит новость (внешний ключ к Users.email)
title	TEXT	Заголовок новости
description	TEXT	Краткое описание статьи
url	TEXT	Ссылка на полную версию статьи
url to image	TEXT	URL изображения
published at	TEXT	Дата и время публикации новости
category	TEXT	Категория новости

3.1.3 Между таблицами Users и SavedNews установлена связь «один ко многим». Один пользователь может сохранить произвольное количество новостей (от нуля до нескольких сотен), и каждая сохранённая новость всегда принадлежит ровно одному пользователю. Данная связь реализована через внешний ключ email в таблице SavedNews, который ссылается на первичный ключ email таблицы Users. При удалении пользователя из таблицы Users каскадно удаляются все связанные с ним записи из таблицы SavedNews (это обеспечивается на уровне логики приложения, так как SQLite по умолчанию не поддерживает автоматическое каскадное удаление без дополнительной настройки внешних ключей).

Таким образом, локальная база данных обеспечивает целостность ссылочных данных и предоставляет модулям приложения унифицированный интерфейс для работы с пользовательской информацией и избранными новостями, что особенно важно для автономного режима работы.

3.2 Облачная база данных Firebase Firestore

Firebase Firestore представляет собой гибкую масштабируемую облачную базу данных, предназначенную для хранения и синхронизации данных между различными устройствами пользователей. В данном проекте Firestore используется в качестве централизованного хранилища, обеспечивающего резервное копирование и доступ к пользовательским данным с любого устройства при наличии подключения к интернету. В отличие от классических реляционных баз данных, Firestore хранит данные в виде коллекций (аналог таблиц) и документов (аналог записей), каждый из которых представляет собой набор полей в формате, близком к JSON. Такой подход обеспечивает высокую гибкость при работе с разнородными данными и упрощает синхронизацию с мобильными клиентами благодаря встроенным возможностям Firebase SDK.

Для управления коллекциями и просмотра содержимого базы данных используется веб-консоль Firebase (Firebase Console). После авторизации и

выбора проекта открывается интерфейс управления, в разделе «Firestore Database» отображаются все созданные коллекции, имеется возможность добавлять новые, редактировать и удалять документы, а также выполнять запросы через встроенную оболочку. На рисунке 3.1 представлен внешний вид консоли Firebase с перечнем коллекций, используемых в приложении «Smart News».

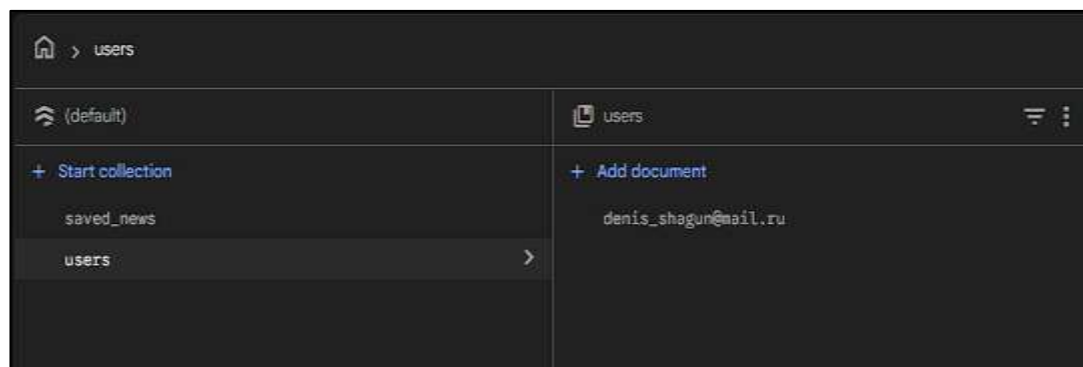


Рисунок 3.1 – Коллекции в Firestore Database

На рисунке 3.2 показан пример содержимого коллекции `saved_news` – документы, соответствующие сохранённым новостям конкретного пользователя.

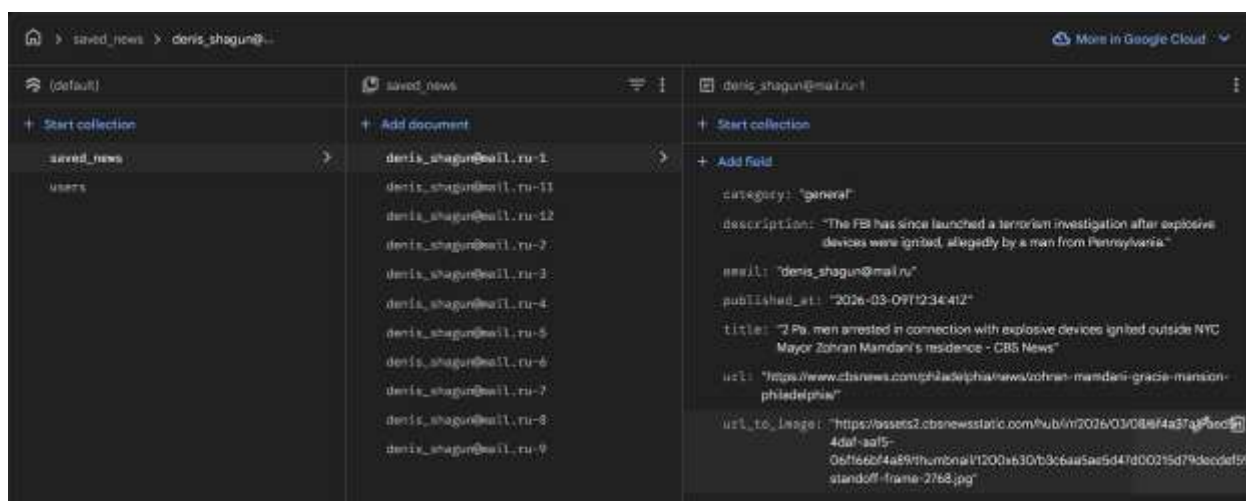


Рисунок 3.2 – Коллекция `saved_news`

Создание коллекций и наполнение их тестовыми данными может выполняться как через веб-интерфейс, так и непосредственно из кода приложения при первом обращении. Ниже приведены описания коллекций, созданных в рамках проекта, и примеры структуры их документов.

Коллекция `users` хранит информацию о зарегистрированных пользователях. Идентификатором документа служит адрес электронной почты пользователя, что обеспечивает простой и быстрый поиск при аутентификации и синхронизации. Каждый документ содержит поля

представлены в таблице 3.3.

Таблица 3.3 – Коллекция `users`

Атрибут	Тип	Описание
<code>name</code>	<code>string</code>	Имя пользователя
<code>email</code>	<code>string</code>	Адрес электронной почты
<code>password</code>	<code>string</code>	Хеш пароля
<code>news_categories</code>	<code>string</code>	Список предпочитаемых категорий новостей
<code>is_vip</code>	<code>boolean</code>	Флаг VIP-статуса
<code>verified</code>	<code>boolean</code>	Флаг подтверждения адреса электронной почты

Коллекция `saved_news` предназначена для хранения новостей, добавленных пользователями в избранное. Документы в этой коллекции имеют составной идентификатор вида `email-newsId`, где `newsId` соответствует локальному идентификатору записи в таблице `SavedNews` на устройстве. Такой подход обеспечивает уникальность документа и упрощает синхронизацию: при сохранении новости на устройстве создается или обновляется соответствующий документ в `Firestore`, а при удалении – удаляется. Каждый документ содержит поля, представленные в таблице 3.4.

Таблица 3.4 – Коллекция `saved_news`

Атрибут	Тип	Описание
<code>email</code>	<code>string</code>	Адрес электронной почты пользователя
<code>title</code>	<code>string</code>	Заголовок новости
<code>description</code>	<code>string</code>	Краткое описание
<code>url</code>	<code>string</code>	Ссылка на полную версию статьи
<code>url to image</code>	<code>string</code>	URL изображения
<code>published at</code>	<code>string</code>	Дата и время публикации новости
<code>category</code>	<code>string</code>	Категория новости

Синхронизация данных между локальной базой `SQLite` и облачной `Firestore` реализована следующим образом:

1 При успешной аутентификации пользователя приложение загружает из `Firestore` все документы коллекции `saved_news`, соответствующие его `email`, и сверяет с локальными данными. Отсутствующие локально записи добавляются, устаревшие – обновляются, а те, которые есть локально, но отсутствуют в облаке (например, созданные в офлайн-режиме), загружаются в `Firestore`.

1 При сохранении новой новости в избранное запись сначала добавляется в локальную таблицу `SavedNews`, а затем асинхронно отправляется в `Firestore`. В случае отсутствия сети операция сохраняется в очереди и выполняется при восстановлении соединения.

2 При удалении новости аналогично удаляется локальная запись и

соответствующий документ в Firestore.

3 Для пользовательских данных (коллекция `users`) синхронизация происходит при регистрации, входе в систему и изменении профиля. Firebase Authentication автоматически управляет состоянием аутентификации, а Firestore используется для хранения дополнительной информации (имя, категории, VIP-статус).

Таким образом, Firebase Firestore обеспечивает надёжное облачное хранение и синхронизацию данных, позволяя пользователям беспрепятственно использовать приложение на нескольких устройствах и восстанавливать информацию после переустановки приложения или смены телефона.

3.3 Модуль управления учетной записью

Модуль управления учетной записью является ключевым компонентом приложения, обеспечивающим безопасный доступ пользователей к системе. Модуль отвечает за регистрацию новых пользователей, аутентификацию при входе, подтверждение адреса электронной почты, восстановление пароля, управление сессией и синхронизацию профильных данных между локальным хранилищем и облаком. В отличие от классических решений с собственным сервером и JWT-токенами, в данном проекте аутентификация полностью делегирована сервису Firebase Authentication, что позволяет не хранить пароли на клиенте и использовать встроенные механизмы безопасности. Для локального хранения данных пользователя применяется SQLite, а для облачной синхронизации – Firestore.

Основными классами, с которыми взаимодействует пользователь, являются `LoginActivity`, `RegisterActivity`, `EmailVerificationActivity` и `ForgotPasswordActivity`. Эти активности реализуют пользовательский интерфейс, выполняют первичную валидацию вводимых данных и вызывают соответствующие методы Firebase Authentication. После успешной аутентификации информация о пользователе сохраняется в локальной базе данных через репозиторий `DatabaseHelper`, а идентификатор пользователя (`USER_ID`) записывается в `SharedPreferences` для автоматического входа при последующих запусках. Модуль также обеспечивает синхронизацию профиля и сохранённых новостей между устройством и облаком, обращаясь к Firestore.

3.3.1 `LoginActivity` представляет собой экран входа в приложение и является одной из точек входа в модуль управлением учетной записью. На этом экране размещены поля для ввода электронной почты и пароля, а также кнопки для выполнения входа, перехода к регистрации и восстановления пароля.

В методе `onCreate` происходит инициализация элементов интерфейса и установка обработчиков нажатий. При нажатии на кнопку входа сначала выполняется проверка заполненности полей и корректности формата электронной почты с использованием регулярного выражения. Если проверка

пройдена, вызывается метод `signInWithEmailAndPassword` из `Firebase Authentication` для аутентификации пользователя. В случае успешного ответа от `Firebase` выполняется получение профиля пользователя из локальной базы данных с помощью `DatabaseHelper.getUserByEmail`. Если пользователь отсутствует в локальной БД (например, после переустановки приложения), его данные загружаются из коллекции `users` `Firestore` и сохраняются локально через `DatabaseHelper.addUser`. Далее идентификатор пользователя сохраняется в `SharedPreferences` под ключом `SAVED_USER_ID` для автоматического входа при последующих запусках, после чего происходит переход на `MainActivity` с передачей `USER_ID` в `Intent`. Если при аутентификации выясняется, что электронная почта не подтверждена, отображается диалог с предложением перейти к верификации и запускается `EmailVerificationActivity`.

3.3.2 `RegisterActivity` реализует экран регистрации новых пользователей. Интерфейс содержит поля для ввода имени, адреса электронной почты и пароля, а также кнопки регистрации и перехода к экрану входа. При нажатии на кнопку регистрации выполняется комплексная валидация введенных данных: проверяется заполненность полей, соответствие электронной почты стандартному шаблону, а также сложность пароля (длина не менее шести символов и наличие хотя бы одного специального символа). Если все проверки пройдены, вызывается метод `createUserWithEmailAndPassword` `Firebase Authentication` для создания учетной записи. При успешном создании пользователя `Firebase` автоматически отправляет письмо для подтверждения электронной почты с помощью `sendEmailVerification`. Одновременно пароль хешируется с использованием `BCrypt`, после чего объект пользователя (имя, email, хеш пароля) сохраняется в локальной базе данных через `DatabaseHelper.addUser`. Для обеспечения синхронизации данные также записываются в `Firestore` в коллекцию `users` с идентификатором документа, равным email пользователя, и полем `verified: false`. После завершения всех операций пользователь перенаправляется на `EmailVerificationActivity` для прохождения процедуры подтверждения.

3.3.3 `EmailVerificationActivity` предназначена для информирования пользователя о необходимости подтвердить адрес электронной почты и управления процессом верификации. На экране отображается сообщение с указанием email, на который отправлено письмо, кнопка повторной отправки письма и прогресс-бар, индицирующий выполнение операций. После создания активности запускается периодическая проверка статуса подтверждения: каждые пять секунд выполняется перезагрузка профиля текущего пользователя `user.reload()` и проверка значения `isEmailVerified`. Как только электронная почта подтверждается, приложение обновляет соответствующий флаг в документе пользователя `Firestore` устанавливает `verified: true`, после чего автоматически перенаправляет пользователя на

главный экран `MainActivity`. Если пользователь нажимает кнопку повторной отправки, вызывается метод `sendEmailVerification`, при успехе отображается диалог с подтверждением, а при ошибке – диалог с описанием проблемы.

3.3.4 `ForgotPasswordActivity` предоставляет интерфейс для восстановления доступа к учётной записи в случае утери пароля. На экране присутствует поле для ввода электронной почты и кнопка отправки запроса на сброс пароля. При нажатии на кнопку вызывается метод `sendPasswordResetEmail` `Firebase Authentication`. В случае успешной отправки письма на указанный адрес отображается диалог с сообщением об успехе, после чего активность завершается, возвращая пользователя на экран входа. Если возникает ошибка (например, указанный email не зарегистрирован в системе), показывается диалог с соответствующим текстом ошибки, и пользователь остаётся на том же экране для повторной попытки.

3.3.5 Для автоматического входа пользователя при повторных запусках приложения используется механизм `SharedPreferences`. После успешной аутентификации в `LoginActivity` идентификатор пользователя сохраняется под ключом `SAVED_USER_ID`. При старте `LoginActivity` в методе `onCreate` выполняется проверка наличия этого ключа: если он существует и в базе данных присутствует пользователь с соответствующим идентификатором, то сразу запускается `MainActivity` с передачей `USER_ID`, минуя экран входа. Выход из системы, реализованный в `ProfileActivity`, приводит к удалению сохранённого идентификатора из `SharedPreferences` и очистке всех данных текущей сессии, после чего пользователь возвращается на `LoginActivity`.

3.4 Модуль профиля

Модуль профиля отвечает за отображение и управление персональными данными текущего пользователя, а также за настройки приложения, связанные с языком интерфейса и валютой. Основные функции модуля включают просмотр информации о пользователе (имя, адрес электронной почты, выбранный язык и валюта), редактирование этих данных, управление платной подпиской (VIP-статус), удаление учётной записи, а также выход из системы. Модуль тесно взаимодействует с локальной базой данных `SQLite` через репозиторий `DatabaseHelper` и с облачным хранилищем `Firestore` для синхронизации изменений, а также использует `SharedPreferences` для хранения настроек языка и валюты.

3.4.1 `ProfileActivity` является основным экраном профиля текущего пользователя. При запуске активности в методе `onCreate` передаётся идентификатор пользователя (`USER_ID`) через `Intent`, на основе которого из `DatabaseHelper` загружается объект `User`. В методе `onCreate` происходит

инициализация элементов интерфейса: текстовых полей для отображения имени, email, языка и валюты, кнопок редактирования каждого поля, кнопок удаления аккаунта, выхода и покупки VIP-подписки, а также контейнера для рекламного баннера. Далее вызывается метод `refreshUI()`, который заполняет текстовые поля актуальными данными: имя и email берутся из объекта `User`, язык – из `SharedPreferences` по ключу `app_language` с последующим преобразованием в читаемое название, валюта – из `SharedPreferences` по ключу `app_currency`. Для кнопок редактирования устанавливаются обработчики, запускающие `EditProfileActivity` с соответствующим параметром (поле для редактирования). Кнопка удаления аккаунта при нажатии отображает диалог подтверждения, после чего вызывается метод `DatabaseHelper.deleteUser`, который удаляет пользователя из локальной базы и инициирует удаление соответствующих данных из `Firebase Authentication` и `Firestore`; при успехе выполняется выход из системы и перенаправление на экран регистрации. Кнопка выхода очищает сохранённый идентификатор пользователя из `SharedPreferences` и завершает сессию `Firebase`, после чего запускает `LoginActivity`. Кнопка покупки VIP открывает `PaymentActivity` для оформления подписки. В методе `updateAdVisibility` реализована логика отображения рекламного баннера: если пользователь не имеет VIP-статуса, загружается и показывается реклама (через `AdView`), а также предусмотрен таймер для повторного показа после закрытия; в противном случае баннер скрыт. В методе `onResume` активность обновляет состояние кнопки VIP и видимость рекламы, а также проверяет, не изменился ли язык приложения, и при необходимости пересоздаёт активность для применения новой локализации.

3.4.2 `EditProfileActivity` предоставляет интерфейс для редактирования персональных данных пользователя. При создании активности в `onCreate` из `Intent` извлекается идентификатор пользователя, после чего инициализируются поля ввода для имени и email, выпадающие списки для выбора языка и валюты, а также соответствующие иконки флагов и валют. Текущие значения подгружаются из `SharedPreferences` и из объекта `User`, полученного через `DatabaseHelper`. Для работы с выпадающими списками используются массивы строк из ресурсов (`R.array.languages` и `R.array.currencies`), адаптеры `ArrayAdapter` с кастомными макетами элементов. При выборе языка или валюты обновляются иконки слева от спиннеров. При нажатии на кнопку сохранения выполняется валидация полей (имя и email не должны быть пустыми), после чего новые значения сохраняются: имя и email обновляются в локальной базе данных через `DatabaseHelper.updateUser`, язык и валюта записываются в `SharedPreferences`. При наличии интернет-соединения изменения синхронизируются с `Firestore`. После успешного сохранения отображается диалог успеха, и пользователь перенаправляется на главный экран (`MainActivity`) с полным сбросом стека активностей, чтобы изменения

локализации применились ко всему приложению. Если валидация не пройдена, показывается диалог ошибки. Кнопка назад возвращает пользователя в профиль без сохранения изменений.

3.5 Модуль новостей

Модуль новостей является центральным компонентом приложения, отвечающим за получение, обработку и отображение актуальных новостных материалов из внешних источников. Основная задача модуля – предоставить пользователю персонализированную ленту новостей, сформированную на основе выбранных категорий, языковых предпочтений и применённых фильтров. Модуль обеспечивает загрузку данных через NewsAPI, их кэширование, отображение в виде списка с возможностью просмотра детальной информации, а также функции сохранения понравившихся статей и их публикации в социальных сетях.

3.5.1 `MainActivity` выступает в роли главного экрана приложения и реализует навигацию между основными разделами через нижнее меню. При запуске активность получает идентификатор пользователя (`USER_ID`) из `Intent`, на основе которого загружается профиль пользователя из базы данных через `DatabaseHelper.getUserById`. В методе `onCreate` выполняется инициализация основных компонентов: настройка `RecyclerView` для отображения списка новостей с использованием `NewsAdapter` и декорации отступов `SpacingItemDecoration`, установка обработчика на кнопку фильтра, запускающего `NewsFilterActivity`, а также настройка нижнего навигационного меню. После получения `email` текущего пользователя вызывается метод `loadNews`, который делегирует загрузку новостей объекту `NewsLoader`. В методе `onResume` активность проверяет изменение языка приложения и при необходимости пересоздаёт себя для применения новой локализации, а также обновляет список новостей. Для обеспечения плавных переходов между активностями используется переопределение анимации через `overridePendingTransition` или `overrideActivityTransition` в зависимости от версии Android.

3.5.2 `NewsAdapter` является адаптером для `RecyclerView`, отвечающим за связь набора новостей с их визуальным представлением. В конструктор адаптера передаётся `email` текущего пользователя, необходимый для сохранения новостей. Метод `onCreateViewHolder` создаёт экземпляр `NewsViewHolder`, наполняет макет `news_item.xml`. Метод `onBindViewHolder` связывает позицию в списке с конкретной новостью, передавая её в метод `bindViewHolder`. Метод `setNews` обновляет внутренний список данных и уведомляет адаптер об изменениях.

`NewsViewHolder` управляет отображением отдельного элемента новости. В его конструкторе находятся ссылки на текстовые поля заголовка,

описания, даты, источника, настроения, а также на изображение и кнопки «Поделиться» и «Сохранить». Метод `bind` заполняет эти поля данными из объекта `News`. Для загрузки изображения используется библиотека `Glide`. Анализ настроения новости выполняется методом `analyzeMood` расширения для `News`, который на основе ключевых слов в тексте определяет эмоциональную окраску (радостная, грустная, нейтральная) и устанавливает соответствующий значок слева от текста настроения. При нажатии на источник открывается браузер с полной статьёй. Кнопка «Поделиться» формирует `Intent` для отправки ссылки. Кнопка «Сохранить» создаёт объект `SavedNews` и через корутину вызывает `DatabaseHelper.saveNews` для сохранения статьи в локальной базе данных и `Firestore`, после чего отображает диалог успеха или ошибки.

3.5.3 `NewsLoader` представляет собой объект с единственным методом `loadNews`, который инкапсулирует всю логику загрузки и фильтрации новостей из `NewsAPI`. Метод выполняется в фоновом потоке с использованием корутин. Сначала проверяется наличие интернет-соединения; при его отсутствии показывается диалог об ошибке. Затем из базы данных извлекается пользователь с указанным `email` и список его предпочитаемых категорий (если не заданы, используется категория «general»). Из `SharedPreferences` читаются настройки фильтрации: язык (определяет также страну для `top-headlines`), даты «от» и «до», выбранное настроение. Для каждой категории формируется запрос: если заданы даты или язык русский, используется `endpoint-everything` с поиском по ключевым словам (переведённым для русского языка), иначе `top-headlines` с параметрами страны и категории. Все полученные статьи объединяются в общий список, после чего удаляются дубликаты по заголовку и применяется фильтр по настроению, если он выбран. Итоговый список передаётся в адаптер на главном потоке. В случае ошибок загрузки или пустого результата отображается соответствующий диалог.

3.6 Модуль фильтрации

Модуль фильтрации предоставляет пользователю возможность тонкой настройки отображаемого новостного контента. Он позволяет выбирать тематические категории, задавать диапазон дат публикации, а также фильтровать новости по их эмоциональной окраске (настроению). Все выбранные параметры сохраняются в `SharedPreferences` и в профиле пользователя (категории), обеспечивая персонализацию ленты при каждом запуске приложения.

3.6.1 `NewsFilterActivity` реализует экран настройки фильтров. При создании активности из `Intent` извлекается идентификатор пользователя, после чего инициализируются элементы интерфейса: `RecyclerView` для

выбора категорий, поля ввода дат «от» и «до», выпадающий список для выбора настроения, кнопки сохранения и сброса фильтров. Для поля даты установлен обработчик нажатия, открывающий `DatePickerDialog`. Выпадающий список настроения заполняется из ресурса `R.array.moods` с помощью `ArrayAdapter`. Текущие значения фильтров загружаются из `SharedPreferences` и устанавливаются в соответствующие элементы.

Для выбора категорий используется `RecyclerView` с адаптером `CategoryAdapter`. Список доступных категорий загружается из профиля пользователя (если ранее были сохранены) и отображается с уже отмеченными элементами. При нажатии на кнопку сохранения формируется строка выбранных категорий (через запятую), которая сохраняется в профиле пользователя с помощью `DatabaseHelper.updateUser`. Значения дат и настроения записываются в `SharedPreferences` под ключами `news_date_from`, `news_date_to` и `news_mood`. После сохранения показывается диалог успеха, и активность завершается с результатом `RESULT_OK`, что сигнализирует `MainActivity` о необходимости перезагрузить новости. Кнопка сброса очищает все выбранные фильтры: снимает отметки с категорий, очищает поля дат, устанавливает настроение «Все», удаляет соответствующие ключи из `SharedPreferences` и обнуляет категории в профиле пользователя.

3.6.2 `CategoryAdapter` служит для отображения списка категорий с флажками выбора. В конструктор передаётся список категорий, множество выбранных категорий и контекст для доступа к строковым ресурсам. Метод `onCreateViewHolder` макет `item_category.xml`, содержащий `CheckBox`. В методе `onBindViewHolder` для каждой категории устанавливается текст (локализованное название) и состояние флажка в соответствии с наличием категории в множестве `selectedCategories`. Обработчик `setOnCheckedChangeListener` при изменении состояния добавляет или удаляет категорию из множества. Метод `getDisplayName` преобразует ключ категории (например, «general») в локализованную строку из ресурсов.

3.7 Модуль категорий новостей

Модуль категорий новостей обеспечивает навигацию по тематическим разделам и отображение сохранённых статей, сгруппированных по категориям. Он предоставляет пользователю удобный интерфейс для просмотра избранных материалов в разрезе каждой категории, что особенно полезно при работе в офлайн-режиме.

3.7.1 `SavedNewsActivity` является точкой входа в раздел сохранённых новостей. При запуске активность получает идентификатор пользователя и отображает список доступных категорий в виде кнопок. В методе `onCreate` инициализируется контейнер для кнопок и нижнее навигационное меню. Для каждой категории из predeterminedного

упорядоченного списка создаётся кнопка с локализованным названием, при нажатии на которую запускается `CategorySavedNewsActivity` с передачей выбранной категории. Метод `refreshCategoryButtons` обновляет список кнопок при изменении языка или повторном открытии активности. В `onResume` также проверяется изменение языка и при необходимости активность пересоздаётся.

3.7.2 `CategorySavedNewsActivity` отображает сохранённые новости конкретной категории. В `onCreate` извлекаются идентификатор пользователя и категория, после чего устанавливается `toolbar` с заголовком, соответствующим выбранной категории. Затем создаётся экземпляр `SavedNewsFragment` с параметрами `userId`, `category` и флагом `isOffline = false` (для онлайн-режима с возможностью удаления), который заменяет контейнер фрагмента. Активность также обрабатывает нажатие на кнопку «Назад» в `toolbar` для завершения.

3.7.3 `SavedNewsFragment` является универсальным фрагментом для отображения списка сохранённых новостей. В методе `onViewCreated` он получает из аргументов `userId`, категорию и флаг `isOffline`. По `userId` через `DatabaseHelper` определяется `email` пользователя, после чего загружается список сохранённых новостей для данной категории. Настраивается `RecyclerView` с `LinearLayoutManager` и декорацией отступов. Создаётся адаптер `SavedNewsAdapter`, которому передаётся список новостей, `email`, слушатель удаления (для обновления списка после удаления) и флаг отображения кнопки удаления (скрывается в офлайн-режиме). При изменении данных (например, после удаления) слушатель вызывает обновление списка через адаптер.

3.7.4 `OfflineCategorySavedNewsActivity` аналогична `CategorySavedNewsActivity`, но предназначена для работы в режиме полного отсутствия интернета (например, при запуске из `NoInternetActivity`). Отличие заключается в том, что фрагменту передаётся `isOffline = true`, что скрывает кнопку удаления (так как удаление в офлайн-режиме может нарушить синхронизацию). В остальном логика совпадает.

3.8 Модуль сохранённых новостей

Модуль сохранённых новостей отвечает за хранение, синхронизацию и отображение статей, добавленных пользователем в избранное. Он обеспечивает доступ к сохранённым материалам как при наличии интернета, так и в офлайн-режиме, а также синхронизирует данные с облачным хранилищем `Firestore`.

3.8.1 `AllSavedNewsActivity` аналогична `SavedNewsActivity`, но используется в контексте офлайн-доступа (например, при отсутствии

интернета пользователь может перейти к сохранённым новостям через кнопку на экране `NoInternetActivity`). Она отображает те же кнопки категорий, но при нажатии открывает `OfflineCategorySavedNewsActivity`. Метод `refreshCategoryButtons` создаёт кнопки для всех категорий, а обработчик нажатия запускает офлайн-активность с выбранной категорией.

3.8.2 `SavedNewsAdapter` адаптирует список сохранённых новостей для отображения в `RecyclerView`. Конструктор принимает список новостей, email пользователя, слушатель удаления и флаг показа кнопки удаления. Метод `onCreateViewHolder` создаёт `SavedNewsViewHolder`, заполняет макет `saved_news_item.xml`. `onBindViewHolder` передаёт данные в `ViewHolder`. Метод `setSavedNews` обновляет список и уведомляет адаптер.

3.8.3 `SavedNewsViewHolder` аналогичен `NewsViewHolder`, но вместо кнопки сохранения содержит кнопку удаления. В методе `bind` заполняются поля заголовка, описания, даты, источника, устанавливается изображение с помощью `Glide`, анализируется настраивается и отображается соответствующий значок. При нажатии на источник открывается браузер. Кнопка «Поделиться» работает так же, как в обычных новостях. Кнопка удаления при нажатии через корутину вызывает `DatabaseHelper.deleteNews` и, в случае успеха, уведомляет слушателя об удалении для обновления списка. Отображение диалогов успеха или ошибки реализовано через вспомогательный метод `showCustomDialog`.

3.9 Модуль нейросетевой рекомендации

Модуль нейросетевой рекомендации реализует ключевую функцию приложения – персонализированные рекомендации новостей на основе анализа сохранённых пользователем статей. Он взаимодействует с серверной частью, написанной на Python с использованием FastAPI и модели BERT, которая выполняет семантический анализ и подбор наиболее релевантных материалов из RSS-лент.

3.9.1 `RecommendActivity` отображает экран с рекомендованными новостями. При запуске из `Intent` извлекается идентификатор пользователя, по которому через `DatabaseHelper` определяется email. В `onCreate` инициализируется `RecyclerView` с `NewsAdapter`, настраивается нижнее навигационное меню, а также контейнер для рекламы. Метод `loadRecommendations` отвечает за загрузку рекомендаций. Он выполняется в фоновом потоке с использованием корутины и предотвращает повторные вызовы с помощью флага `isLoadingRecommendations`. Сначала из базы данных извлекаются все сохранённые новости пользователя (по всем категориям). Затем формируется JSON-объект с этими новостями и отправляется POST-запросом на серверный endpoint `/recommend`. Для

выполнения запроса используется `OkHttp`. В ответ сервер возвращает список новостей (в формате JSON), который парсится в объекты `News` и передаётся в адаптер на главном потоке. В случае ошибки или пустого ответа отображается диалог с соответствующим сообщением. Метод `onResume` проверяет изменение языка и при необходимости пересоздаёт активность. Рекламный баннер управляется аналогично другим активностям через `updateAdVisibility`.

3.9.2 Серверная часть модуля реализована на `FastAPI` и включает класс `GlobalRecommender`, который инкапсулирует логику загрузки модели BERT, построения пользовательского профиля и подбора рекомендаций. При инициализации загружается предобученная мультязычная модель `bert-base-multilingual-uncased` и токенизатор. Для каждого сохранённого новостного элемента извлекается эмбединг (усреднённое значение последнего скрытого слоя BERT), а также `one-hot` векторы категории и настройки. Все признаки объединяются, нормализуются и усредняются, формируя пользовательский профиль. Кандидаты для рекомендаций собираются из RSS-лент (английских и русских) по семи категориям. Для каждого кандидата также вычисляются признаки, после чего вычисляется косинусная близость с профилем пользователя. Возвращаются 20 наиболее похожих новостей. Эндпоинт `/recommend` принимает список сохранённых новостей, вызывает построение профиля, загружает кандидатов и возвращает рекомендации в формате JSON.

3.10 Модуль рекламы и платной подписки

Модуль рекламы и платной подписки отвечает за монетизацию приложения. Он интегрирован в основные активности (`MainActivity`, `ProfileActivity`, `RecommendActivity`) и управляет отображением рекламных баннеров для бесплатных пользователей, а также предоставляет интерфейс для оформления VIP-подписки, отключающей рекламу и дающей дополнительные возможности.

3.10.1 `PaymentActivity` реализует экран ввода платёжных данных для оформления VIP-подписки. Интерфейс содержит поля для номера карты, имени держателя, месяца и года истечения срока, а также CVV-кода. При нажатии на кнопку оплаты выполняется простая валидация (длина полей соответствует стандартам платёжных карт). При успешной валидации вызывается метод `DatabaseHelper.updateUser`, который устанавливает флаг `isVip` в `true` для текущего пользователя, после чего изменения синхронизируются с `Firestore`. Затем отображается диалог успеха, и активность завершается, возвращая пользователя в профиль с обновлённым статусом VIP. В случае ошибки валидации или обновления данных показывается диалог ошибки. Кнопка назад возвращает в профиль.

4 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ

5 ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ

6 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

7 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ И РЕАЛИЗАЦИИ НА РЫНКЕ МОБИЛЬНОГО ПРИЛОЖЕНИЯ ДЛЯ НОВОСТЕЙ С ПОДДЕРЖКОЙ НЕЙРОСЕТЕВЫХ АЛГОРИТМОВ РЕКОМЕНДАЦИЙ

7.1 Характеристика программного средства, разрабатываемого для реализации на рынке

Дипломный проект представляет собой нативное приложение для ОС Android, которое предоставляет пользователям возможность искать актуальные новости, сохранять их в избранное и делиться новостями с друзьями, а также читать новости, основаны на предпочтениях пользователя.

Цель разработки данного проекта – предоставление новой удобной площадки для поиска и ознакомления актуальных новостей. Приложение предназначено для использования в образовательных целях.

Целевой аудиторией данного приложения может быть человек любого возраста, которого интересует текущие новости.

На момент разработки найдено несколько приложений со схожим функционалом: Google Новости, Flipboard, Apple News+ и несколько веб-решений, все эти варианты описаны в обзоре литературы. Веб-решения имеют достаточно устаревший интерфейс и не всем доступны. Flipboard удобен и прагматичен в оформлении, но основная аудитория иноязычная. Функционал дипломного проекта в каких-то моментах схож с существующим решением, однако не в полной мере. Отличительной особенностью дипломного проекта является индикатор настроения новостей. Такой индикатор добавляет смайлик эмоции, по которому пользователь может понять эмоциональный окрас новости.

7.2 Расчёт инвестиций в разработку программного средства

7.2.1 Расчёт зарплат на основную заработную плату разработчиков. Расчёт затрат на основную заработную плату разработчиков производится исходя из количества людей, которые занимаются разработкой программного продукта, месячной зарплаты каждого участника процесса разработки и сложности выполняемой ими работы.

В разработке приложения принимают участие: UI/UX дизайнер, инженер-программист, инженер-тестировщик и интернет-маркетолог.

UI/UX специалист разрабатывает модели и сценарии взаимодействия пользователя с интерфейсом. Данный специалист разрабатывает прототипы интерфейса.

Задачами инженера-программиста являются создание модели данных, графического интерфейса, связи между моделью данных и графическим интерфейсом, создание и администрирование базы данных, и обработка данных из базы.

Инженер-тестировщик занимается выявлением неработоспособных

частей приложения, а также оценивает пользовательский опыт, получаемый от использования приложения.

Интернет-маркетолог обеспечивает рекламу продукта, находит определенных пользователей, которым будет интересно данное приложение и предлагает им ознакомиться с ним.

Затраты на основную заработную плату рассчитаны по формуле 7.1:

$$Z_o = K_{\text{пр}} \sum_{i=1}^n Z_{\text{чи}} \cdot t_i, \quad (7.1)$$

где $K_{\text{пр}}$ — коэффициент премий и иных стимулирующих выплат;

n — категории исполнителей, занятых разработкой программного средства;

$Z_{\text{чи}}$ — часовая заработная плата исполнителя i -й категории, р;

t_i — трудоемкость работ, выполняемых исполнителем i -й категории, ч.

Месячная заработная плата для специалистов, участвующих в разработке и продвижении специализированного приложения в Минске в 2025 году, определена на основе текущих рыночных условий и уровня квалификации. Для Junior Android инженера-программиста установлен оклад в размере 450 долларов США в месяц [15], что соответствует уровню начальной позиции в сфере разработки мобильных приложений. Для Junior инженера-тестировщика, обеспечивающего контроль качества продукта, заработная плата составляет 350 долларов США [16]. UI/UX дизайнер, ответственный за создание удобного и привлекательного интерфейса, получает 400 долларов США в месяц [17]. Особое внимание в проекте уделяется продвижению приложения, так как оно является узконаправленным и зависит от активности пользователей, которые формируют контент платформы. Для обеспечения эффективной рекламной кампании привлекается опытный интернет-маркетолог с месячным окладом в 650 долларов США [18].

По состоянию на 18 февраля 2026 года, 1 доллар США по курсу Национального Банка Республики Беларусь составляет 2,86 белорусских рублей. В перерасчёте на белорусские рубли месячные оклады для инженера-программиста, инженера-тестировщика, ui/ux дизайнера и интернет-маркетолога составляют 1287,00, 1001,00, 1144,00 и 1859,00 белорусских рублей соответственно.

Часовой оклад исполнителей высчитывается путём деления их месячного оклада на количество рабочих часов в месяце. На разработку дизайна и прототипирования приложения ui/ux дизайнеру потребуется 2 недели, а это 80 часов. На разработку приложения уйдет около 2 месяцев поэтому для инженера программиста значение часов равно 320 так же, как и для инженера-тестировщика. Интернет-маркетолог начнет работу во время разработки и продолжит еще месяц после релиза продукта, в итоге выйдет около 2 месяцев, то есть так же 320 часов. Коэффициент премии приравнивается к единице, так как она входит сумму заработной платы.

Затраты на основную заработную плату приведены в таблице 7.1.

Таблица 7.1 – Затраты на основную заработную плату

Категория исполнителя	Месячный оклад, р	Часовой оклад, р	Трудоёмкость работ, ч	Итого, р
Инженер-программист	1287,00	7,67	320	2454,40
Инженер-тестировщик	1001,00	5,96	320	1906,67
UI/UX дизайнер	1144,00	6,81	80	544,80
Интернет-маркетолог	1859,00	11,07	320	3542,40
Итого				8448,27
Премия и иные стимулирующие выплаты (0%)				0
Всего затраты на основную заработную плату разработчиков				8448,27

7.2.2 Расчёт затрат на дополнительную заработную плату разработчиков рассчитывается по формуле 7.2:

$$З_д = \frac{З_о \cdot Н_д}{100}, \quad (7.2)$$

где $Н_д$ – норматив дополнительной заработной платы.

Значение норматива дополнительной заработной платы принимает 10 %.

7.2.3 Размер отчислений на социальные нужды определяется согласно ставке отчислений равняется 34,6.

Расчёт отчислений на социальные нужды вычисляется по формуле 7.3:

$$Р_{соц} = \frac{(З_о + З_д) \cdot Н_{соц}}{100}, \quad (7.3)$$

где $Н_{соц}$ – норматив отчислений в ФСЗН и Белгосстрах.

7.2.4 Расчёт затрат на прочие расходы определяется при помощи норматива прочих расчётов. Эта величина имеет значение 30%. Расчёт прочих расходов вычисляется по формуле 7.4:

$$Р_{пр} = \frac{З_о \cdot Н_{пр}}{100}, \quad (7.4)$$

где $Н_{пр}$ – норматив прочих расходов.

7.2.5 Для того, чтобы рассчитать расходы на реализацию, необходимо знать норматив расходов на неё. Принимаем значение норматива равным 3%. Формула, которая используется для расчёта расходов на реализацию, имеет следующий вид:

$$P_p = \frac{Z_o \cdot H_p}{100}, \quad (7.5)$$

где H_p – норматив расходов на реализацию.

7.2.6 Расчёт общей суммы затрат на разработку и реализацию. Определяем общую сумму затрат на разработку и реализацию как сумму ранее вычисленных расходов: на основную заработную плату разработчиков, дополнительную заработную плату разработчиков, отчислений на социальные нужды, расходы на реализацию и прочие расходы. Значение определяется по формуле 7.6:

$$Z_p = Z_o + Z_d + P_{\text{соц}} + P_{\text{пр}} + P_p, \quad (7.6)$$

Таким образом, величина затрат на разработку программного средства высчитывается по указанной выше формуле и указана в таблице 7.2:

Таблица 7.2 – Затраты на разработку

Название статьи затрат	Формула/таблица для расчёта	Значение, р.
Основная заработная плата разработчиков	См. таблицу 7.1	8448,27
Дополнительная заработная плата разработчиков	$Z_d = \frac{8448,27 \cdot 10}{100}$	844,827
Отчисление на социальные нужды	$P_{\text{соц}} = \frac{(8448,27 + 844,827) \cdot 34,6}{100}$	3215,41
Прочие расходы	$P_{\text{пр}} = \frac{8448,27 \cdot 30}{100}$	2534,48
Расходы на реализацию	$P_p = \frac{8448,27 \cdot 3}{100}$	253,448
Общая сумма затрат на разработку и реализацию	$Z_p = 8448,27 + 844,827 + 3215,41 + 2534,48 + 253,448$	15296,467

7.3 Расчёт экономического эффекта от реализации программного средства на рынке

Экономический эффект организации-разработчика программного средства представляет собой прирост чистой прибыли от его продажи на рынке потребителям, величина которого зависит от объема продаж, цены реализации и затрат на разработку программного средства.

Для подсчета примерного количества пользователей можно рассмотреть приложение-аналог Flipboard, которое было выпущено в 2010 году и на данный момент насчитывает около 145 миллионов активных пользователей. Однако большинство пользователей – иноязычные, что затрудняет получение корректной статистики. На платформе Apple News+ зарегистрировано несколько миллионов пользователей, что указывает на потенциальное количество людей, которые могут активно взаимодействовать с контентом. Google Новости, в свою очередь, имеет многомиллионную аудиторию и также является международным приложением. Основная проблема заключается в том, что не все пользователи готовы приобретать платную подписку для отключения рекламы в бесплатной версии приложения. Это может привести к снижению доходов на начальном этапе, так как многие предпочтут использовать приложение с рекламой. Однако инвестиции в работу интернет-маркетолога помогут смягчить эту ситуацию, например, за счет привлечения пользователей из сторонних приложений или размещения таргетированной рекламы в социальных сетях, что повысит узнаваемость приложения и стимулирует рост числа пользователей. Обобщив числа по другим приложениям, предположим, что за год количество пользователей достигнет 100 тысяч.

Приложение можно выложить в Google Play взывая небольшую плату за подписку. Учитывая вышеописанные факторы наиболее оптимальной ценой данного приложения, предполагается 1,99 Долларов США, что с вычетом комиссии Google Play в 30% составляет 1,4 Доллара США. Таким образом, отпускная цена подписка программного средства составляет 4,01 Белорусских рубля. После пробного периода такая небольшая цена не должна стать помехой и причиной ухода пользователя, так как она остается доступной для большинства потенциальных пользователей и соответствует среднему уровню стоимости цифровых подписок.

Для расчёта прироста чистой прибыли необходимо учесть налог на добавленную стоимость, который высчитывается по следующей формуле 7.7:

$$\text{НДС} = \frac{\text{Ц}_{\text{отп}} \cdot N \cdot \text{Н}_{\text{д.с}}}{100\% + \text{Н}_{\text{д.с}}}, \quad (7.7)$$

где N – количество копий(лицензий) программного продукта, реализуемое за год, шт.;

$\text{Ц}_{\text{отп}}$ – отпускная цена копии программного средства, р.;

$\text{Н}_{\text{д.с}}$ – ставка налога на добавленную стоимость, %.

Ставка налога на добавленную стоимость по состоянию на 18 февраля 2026 года, в соответствии с действующим законодательством Республики Беларусь, составляет 20%. Используя данное значение, посчитаем НДС:

$$\text{НДС} = \frac{4,01 \cdot 100\,000 \cdot 20\%}{100\% + 20\%} = 66833,33 \text{ р.}$$

Посчитав налог на добавленную стоимость, можно рассчитать прирост чистой прибыли, которую получит разработчик от продажи программного продукта. Для этого используется формула 7.8:

$$\Delta\Pi_{\text{ч}}^{\text{p}} = (\text{Ц}_{\text{отп}} \cdot N - \text{НДС}) \cdot \text{Р}_{\text{пр}} \cdot \left(1 - \frac{\text{Н}_{\text{п}}}{100}\right), \quad (7.8)$$

где N – количество копий(лицензий) программного продукта, реализуемое за год, шт.;

$\text{Ц}_{\text{отп}}$ – отпускная цена копии программного средства, р.;

НДС – сумма налога на добавленную стоимость, р.;

$\text{Н}_{\text{п}}$ – ставка налога на прибыль, %;

$\text{Р}_{\text{пр}}$ – рентабельность продаж копий.

Ставка налога на прибыль, согласно действующему законодательству, равна 20%. Рентабельность продаж копий взята в размере 20%. Зная ставку налога и рентабельность продаж копий (лицензий), рассчитывается прирост чистой прибыли для разработчика:

$$\Delta\Pi_{\text{ч}}^{\text{p}} = (4,01 \cdot 100\,000 - 66833,33) \cdot 20\% \cdot \left(1 - \frac{20}{100}\right) = 53\,466,67 \text{ р.}$$

7.4 Расчёт показателей экономической эффективности разработки и реализации программного средства на рынке

Для того, чтобы оценить экономическую эффективность разработки и реализации программного средства на рынке, необходимо рассмотреть результат сравнения затрат на разработку данного программного продукта, а также полученный прирост чистой прибыли за год.

Сумма затрат на разработку меньше суммы годового экономического эффекта, поэтому можно сделать вывод, что такие инвестиции окупятся менее, чем за один год.

Таким образом, оценка экономической эффективности инвестиций производится при помощи расчёта рентабельности инвестиций (Return on Investment, ROI) по формуле 7.9:

$$ROI = \frac{\Delta\Pi_{\text{ч}}^{\text{p}} - \text{З}_{\text{р}}}{\text{З}_{\text{р}}} \cdot 100\% \quad (7.9)$$

где $\Delta\Pi_{\text{ч}}^{\text{p}}$ – прирост чистой прибыли, полученной от реализации программного средства на рынке информационных технологий, р.;

$\text{З}_{\text{р}}$ – затраты на разработку и реализацию программного средства, р.

$$ROI = \frac{53466,67 - 15296,467}{15296,467} \cdot 100\% = 269,00\% \quad (7.10)$$

7.5 Вывод об экономической целесообразности реализации проектного решения

Проведённые расчёты технико-экономического обоснования позволяют сделать предварительный вывод о целесообразности разработки данного программного продукта. Общая сумма затрат на разработку и реализацию составила 15296,467 белорусских рублей, а отпускная цена была установлена на уровне 4,06 белорусских рублей.

Прирост чистой прибыли за год, исходя из предполагаемого объёма продаж в размере 100 000 версий в год, составляет 53466,67 белорусских рублей. Рентабельность инвестиций за год составляет 269,00%.

Это означает, что разработка данного программного продукта является целесообразной и реализация программного средства по установленной цене имеет смысл.

Однако следует учитывать возможные риски, связанные с тем, что не все пользователи будут приобретать платную подписку для отключения рекламы в бесплатной версии приложения. Это может привести к снижению доходов на начальном этапе и замедлению монетизации проекта. Тем не менее, при грамотной маркетинговой стратегии, включающей таргетированную рекламу и привлечение аудитории из сторонних платформ, со временем проект способен достичь устойчивого роста и успешного развития.

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Официальный сайт Google [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://blog.google/products-and-platforms/products/> – Дата доступа: 17.02.2026.
- [2] Официальный сайт Apple News+ [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://www.apple.com/apple-news/> – Дата доступа: 17.02.2026.
- [3] Официальный сайт Flipboard [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://about.flipboard.com/how-it-works/> – Дата доступа: 17.02.2026.
- [4] Билл Филлипс, Крис Стюарт, Кристин Марсикано. Android. Программирование для профессионалов. – СПб.: Питер, 2021. – 768 с.
- [5] Документация Kotlin Language [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://kotlinlang.org/docs/home.html> – Дата доступа: 17.02.2026.
- [6] Документация FastAPI [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://fastapi.tiangolo.com/> – Дата доступа: 17.02.2026.
- [7] Официальная документация сервиса NewsAPI [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://newsapi.org/docs> – Дата доступа: 17.02.2026.
- [8] Официальный сайт среды разработки Android Studio [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://developer.android.com> – Дата доступа: 17.02.2026.
- [9] Официальная документация Android Developers. Layout Editor [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://developer.android.com/studio/write/layout-editor> – Дата доступа: 17.02.2026.
- [10] Документация Firebase [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://firebase.google.com/docs> – Дата доступа: 17.02.2026.
- [11] Информационный ресурс по различным языкам и технологиям программирования, компьютерам, мобильным платформам [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://metanit.com/java/android/2.1.php> – Дата доступа: 17.02.2026.
- [12] Ричардсон, К. Микросервисы. Паттерны разработки и рефакторинга. – СПб.: Питер, 2019. – 544 с.
- [13] Информационный ресурс, посвященный мобильной разработке [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://apptractor.ru/info/articles/retrofit.html> – Дата доступа: 17.02.2026.
- [14] MVC [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://developer.mozilla.org/en-US/docs/Glossary/MVC> – Дата доступа: 17.02.2026.
- [15] Сайт вакансий rabota.by [Электронный ресурс]. – Электронные

данные. – Режим доступа: <https://rabota.by/vacancies/junior-android-developer> – Дата доступа 17.02.2026.

[16] Сайт вакансий rabota.by [Электронный ресурс]. – Электронные данные. – Режим доступа: https://rabota.by/vacancies/inzhener_po_testirovaniyu_po – Дата доступа 17.02.2026.

[17] Сайт вакансий rabota.by [Электронный ресурс]. – Электронные данные. – Режим доступа: <https://rabota.by/vacancies/ui-ux-designer> – Дата доступа 17.02.2026.

[18] Сайт вакансий rabota.by [Электронный ресурс]. – Электронные данные. – Режим доступа: https://rabota.by/vacancies/internet_marketolog – Дата доступа 17.02.2026.

Вычислительные машины, системы и сети: дипломное проектирование [Электронный ресурс]. – Электронные данные. – Режим доступа: https://www.bsuir.by/m/12_100229_1_185678.pdf – Дата доступа: 13.03.2026.

Экономика проектных решений: методические указания по экономическому обоснованию дипломных проектов [Электронный ресурс]. – Электронные данные. – Режим доступа: https://libeldoc.bsuir.by/bitstream/123456789/45990/1/Gorovoi_2021.pdf – Дата доступа: 13.03.2026.

ПРИЛОЖЕНИЕ А
(обязательное)

Вводный плакат. Плакат

ПРИЛОЖЕНИЕ Б
(обязательное)

Мобильное приложение для новостей. Схема структурная

ПРИЛОЖЕНИЕ В
(обязательное)

Мобильное приложение для новостей. Диаграмма классов

ПРИЛОЖЕНИЕ Г
(обязательное)

Мобильное приложение для новостей. Диаграмма последовательности

ПРИЛОЖЕНИЕ Д
(обязательное)

Мобильное приложение для новостей. Схема программы

ПРИЛОЖЕНИЕ Е
(обязательное)

Заключительный плакат. Плакат